

Comprehensive DVA-C02 Project Dossier: Advanced CI/CD Deployments

Engineer: AWS DevOps Candidate

Target Certification: AWS Certified Developer - Associate (DVA-C02)

Project Scope: Implementation of EC2 Blue/Green and Serverless Canary deployment pipelines via AWS CodeDeploy, integrating IAM security boundaries and CloudWatch automated rollbacks.

1. Executive Summary & Architecture

This project successfully validated the ability to architect, deploy, and troubleshoot zero-downtime deployment pipelines in AWS.

Architecture 1: EC2 Blue/Green via CodeDeploy

- **Infrastructure:** Application Load Balancer (ALB) routing traffic to Target Groups (TG-Blue and TG-Green), backed by Auto Scaling Groups (ASGs).
- **Compute:** Amazon Linux 2023 EC2 instances bootstrapped with the CodeDeploy Agent via Launch Template User Data.
- **Cutover Strategy:** CodeDeployDefault.AllAtOnce with ALB health-check validation before traffic rerouting.

Architecture 2: Serverless Canary via AWS Lambda

- **Infrastructure:** AWS Lambda function utilizing immutable Versions and Aliases (prod).
- **Cutover Strategy:** CodeDeployDefault.LambdaCanary10Percent10Minutes.
- **Resilience:** CloudWatch Alarms tracking the Errors metric, triggering automated pipeline rollbacks if Errors >= 1.

2. The Error Crucible: Real-World Troubleshooting Log

The highest value of this project was the real-world debugging. The DVA-C02 exam tests your ability to identify *why* a pipeline failed. Below is the exact log of errors encountered during this sprint, their root causes, and the engineering fixes applied.

Error 1: The AWS CLI Syntax & Placeholder Trap

- **Symptom:** argument subcommand: Found invalid choice 'api' and uploading code to a literal bucket named your-unique-codedeploy-bucket-name.
- **Root Cause:** Misunderstanding the distinction between high-level S3 commands and low-level S3 API commands, coupled with blindly copy-pasting documentation placeholders.
- **The Fix:** Using the correct aws s3api command for versioning and explicitly defining the target bucket (projects-aws-bucket).
- **DVA-C02 Lesson:** Always use s3api for bucket configurations (versioning, CORS, policies). High-level s3 is strictly for file transfers (cp, mv, sync).

Error 2: The Auto Scaling Cloning Block

- **Symptom:** The IAM role... does not give you permission to perform operations in the following AWS service: AmazonAutoScaling.

- **Root Cause (The Legacy Trap):** CodeDeploy was trying to clone an ASG that relied on a modern *Launch Template*. The default `AWSCodeDeployRole` managed policy only natively supports deprecated *Launch Configurations*.
- **The Fix:** Appending an inline policy to the CodeDeploy Service Role granting `ec2:RunInstances` and `ec2:CreateTags`.
- **DVA-C02 Lesson:** CodeDeploy requires deep EC2 permissions to facilitate Blue/Green infrastructure cloning when using modern Launch Templates.

Error 3: The `iam:PassRole` Authorization Failure

- **Symptom:** Auto Scaling failed to launch the Green instances on behalf of CodeDeploy.
- **Root Cause:** CodeDeploy was attempting to attach the `EC2-S3-CodeDeploy-Role` to the newly launched instances but lacked the authority to hand off that role.
- **The Fix:** Adding the `iam:PassRole` action to the CodeDeploy Service Role.
- **DVA-C02 Lesson:** Anytime an AWS service (like CodeDeploy or CloudFormation) creates a resource (like EC2) that requires an IAM role, the creating service *must* have the `iam:PassRole` permission.

Error 4: The CodeDeploy Agent Timeout

- **Symptom:** `CodeDeploy agent was not able to receive the lifecycle event.`
- **Diagnostics Run:** SSH'd into the instance. Checked `/var/log/cloud-init-output.log` and ran `sudo systemctl status codedeploy-agent`.
- **Root Cause:** The agent successfully installed and was running a `200 OK` poll to AWS, but the instance took too long to boot and install dependencies. The AWS control plane timed out the deployment window before the agent's first check-in.
- **The Fix:** Triggering a fresh deployment while the agent was already active and polling.
- **DVA-C02 Lesson:** CodeDeploy operates on a long-polling architecture. If instances lack internet access (no Public IP or NAT Gateway in the VPC) or take too long to boot, CodeDeploy will register a timeout failure.

Error 5: The `UnknownError` at `BeforeInstall`

- **Symptom:** Deployment instantly failed at the `BeforeInstall` hook in 0 seconds with an `UnknownError`. `DownloadBundle` succeeded.
- **Root Cause:** The `zip` command failed to include the AppSpec file because of a typo (`appspec.yaml` instead of `appspect.yaml`). The CodeDeploy agent extracted the archive, couldn't find the exact instruction file at the root, and crashed.
- **The Fix:** Renaming the file to exactly `appspect.yaml`, re-zipping from *inside* the project directory, and re-uploading to S3.
- **DVA-C02 Lesson:** The AppSpec file must be named perfectly (`appspect.yaml`) and must reside at the absolute root level of the revision archive.

Error 6: The "Alias Cannot Be Found" Lambda Error

- **Symptom:** CodeDeploy threw an error stating the Lambda function or alias could not be found, despite it existing in the console.
- **Root Cause:** Case-sensitivity mismatch. The YAML AppSpec requested Alias `PROD`, but the actual AWS Lambda console alias was named `prod`.
- **The Fix:** Updating the CodeDeploy configuration/AppSpec to match the exact lowercase string `prod`.

- **DVA-C02 Lesson:** AppSpec definitions (Function Name, Alias Name) are strictly case-sensitive. Additionally, CodeDeploy and Lambda resources must exist in the exact same AWS Region.

3. Core Concepts Mastered for the DVA-C02

To secure a top score, remember these architectural rules reinforced by this project:

Domain	Rule / Concept
IAM Separation	Service Roles dictate what the AWS orchestrator (CodeDeploy) can touch. Instance Profiles dictate what the compute unit (EC2) can touch (like downloading from S3).
AppSpec Hooks (EC2)	Hook scripts (BeforeInstall , ValidateService) execute <i>locally</i> on the EC2 instance via the Ruby agent. Network hooks (AllowTraffic) execute <i>externally</i> on the AWS control plane.

Target Group Registration	Never manually register instances in an ASG CI/CD pipeline. Attach the Target Group to the ASG configuration and let AWS handle the dynamic IP registration.
Lambda Versioning	CodeDeploy cannot manipulate the \$LATEST Lambda version. You must explicitly publish immutable Versions and use Aliases to execute traffic shifts.
Automated Rollbacks	CloudWatch Alarms evaluate metric thresholds (e.g., Errors >= 1). CodeDeploy intercepts the ALARM state transition to trigger immediate traffic reversion, preventing prolonged downtime.